Comparative Study on Custom and Package-Ready Implementations of the K-Nearest Neighbours (kNN) Algorithm

39993035

2026-01-21

1 Abstract

This report presents a systematic comparison between a custom k-nearest neighbors (kNN) implementation and a widely used package-based alternative across multiple benchmark datasets. The evaluation framework combines repeated train–test experiments with Monte Carlo permutation tests to assess whether observed differences between implementations are statistically significant. Two data splits (70/30 and 80/20) are considered to examine robustness with respect to training set size. Model behavior is evaluated using a diverse set of metrics, including macro-averaged classification scores, probabilistic loss functions, calibration measures, and fine-grained timing components such as data loading, training, and testing.

Paired Monte Carlo permutation tests are employed to quantify distributional differences in performance metrics, complemented by Wilcoxon signed-rank tests as a non-parametric robustness check. The results indicate that standard predictive metrics on held-out test sets are generally indistinguishable between the two implementations, suggesting equivalent classification capability. However, several cross-validated loss and calibration metrics exhibit statistically significant differences, highlighting the influence of implementation-specific design choices on probabilistic outputs. Timing analyses further reveal consistent disparities in computational efficiency, particularly in stages involving repeated model evaluation.

Overall, the findings show that while predictive performance may appear similar when judged by conventional accuracy-oriented metrics, deeper statistical analysis uncovers meaningful differences in efficiency and calibration. These results emphasize the value of Monte Carlo permutation testing as a rigorous tool for implementation-level comparison and demonstrate that algorithmic equivalence does not necessarily imply identical practical behavior.

2 Introduction

Artificial intelligence (AI) and machine learning (ML) have become central to modern data analysis and decision-making across a wide range of domains, including healthcare, finance, cybersecurity, and scientific research (Dritsas and Trigka 2025). The increasing availability of large-scale datasets, together with advances in computational infrastructure, has accelerated the adoption of learning-based methods, with classification tasks representing a substantial proportion of scientific research articles (Guidotti, Pandolfo, and Pulina 2025), (Ersozlu, Taheri, and Koch 2024). Despite the rapid development of complex models such as deep neural networks and ensemble methods, classical machine learning algorithms remain widely used due to their interpretability, computational efficiency, and well-understood theoretical properties (Mehdiyev, Majlatow, and Fettke 2025). As a result, these methods continue to serve as important benchmarks in empirical studies and applied research.

Classification is a fundamental problem in supervised learning, where the objective is to assign observations to one of several predefined categories based on their features (Karakullukcu 2025). Numerous classification methods have been proposed in the literature, differing in their assumptions, complexity, and practical implementation. The k-Nearest Neighbours (kNN) algorithm is one such method that has

been widely studied and applied in a variety of classification settings (Alshammari, Bencsik, and Ali 2026). Its continued use in both academic and applied contexts has led to the availability of multiple implementations across software platforms.

In practical applications, kNN is commonly implemented using established machine learning packages that provide standardised interfaces and specialised computational routines. Alternatively, custom implementations may be developed for experimental, educational, or analytical purposes. Differences between these implementation approaches may arise due to design choices, numerical precision, or computational strategies, even when the underlying algorithmic concept is held constant.

The aim of this study Is to conduct an empirical comparison between a package-based implementation of the k-Nearest Neighbours classifier and a custom-built implementation developed from first principles. The comparison is carried out under controlled experimental conditions, using identical data splits, fixed algorithmic parameters, and consistent evaluation procedures. Model behaviour is evaluated using a range of performance, probabilistic, and calibration metrics, alongside measures of computational efficiency such as execution time and memory usage. Statistical hypothesis testing is applied to assess whether any observed differences between implementations are statistically significant.

3 Background

This section of the report is dedicated to explaining the theoretical background of the k-Nearest Neighbours, leaving out details about processes such as data splitting, cross-validation, and performance evaluation. The main reason for keeping this section shorter than intended is to keep the report as short as possible, falling within the allowed limit of 13 pages.

3.1 Overview of kNN

The k-Nearest Neighbours (k-NN) algorithm is a non-parametric, instance-based classification method that, compared to model-based learning approaches, does not rely on training data to make decisions, thus, it is considered a so-called "lazy learner" (Halder et al. 2024). Classification is performed using local decision rules, where the label of a query point is inferred from the empirical distribution of labels among its nearest neighbours (Gweon, Schonlau, and Steiner 2019b). Unlike parametric classifiers, k-NN does not assume a fixed functional form for class boundaries, enabling flexible decision surfaces but at the cost of increased computational complexity at inference time (Alshammari, Bencsik, and Ali 2026). Known strengths of k-NN include conceptual simplicity and strong asymptotic properties, while its limitations include sensitivity to distance metrics, feature scaling, and high-dimensional data (Abu Alfeilat et al. 2019), (Lu and Gweon 2025).

3.2 Logical Structure of the kNN Algorithm

This subsection, as the name suggests, focuses on capturing the necessary theoretical background of the logical flow of the k-NN algorithm, giving valuable insights into the implementation that are going to be helpful in the process of actually designing the final model.

3.2.1 Problem Formulation and Feature Space Representation

The k-Nearest Neighbours (kNN) algorithm operates within a supervised multi-class classification framework, assuming a labelled dataset drawn from an unknown joint distribution (Gweon, Schonlau, and Steiner 2019a). Observations are embedded in a d-dimensional feature space endowed with a metric structure. Classification relies on the locality hypothesis, which assumes that proximity in feature space corresponds to similarity in class membership (Mehta et al. 2018). Consequently, the effectiveness of k-NN depends critically on both the quality of feature representation and the geometry induced by the chosen distance metric.

3.2.2 Distance Metrics and Induced Geometry

Distance metrics define neighbourhood structure and induce the geometry of the feature space. Euclidean distance is the most common choice and is appropriate when features are continuous and similarly scaled (Mehta et al. 2018). Alternatives such as Manhattan distance and the more general Minkowski family allow for different neighbourhood shapes and sensitivity to coordinate-wise deviations. Feature scaling is therefore essential, as unscaled variables can dominate distance computations. In high-dimensional settings, distance concentration reduces the contrast between neighbours, exposing kNN to the curse of dimensionality and degrading local discrimination (Aggarwal, Hinneburg, and Keim 2001).

3.2.3 Neighbour Selection and Locality Control

Neighbour selection is performed by computing distances between a query point and all training samples, ranking them, and selecting the k closest instances. The parameter k determines the degree of locality and induces a bias–variance trade-off (Halder et al. 2024). Small values of k yield highly local decision rules with low bias but high variance, while larger values smooth decision boundaries at the cost of increased bias. The choice of k thus directly affects predictive stability and generalisation performance.

3.2.4 Voting and Probability Estimation

Class prediction is typically obtained through majority voting among the selected neighbours. Uniform voting assigns equal weight to each neighbour, whereas distance-weighted schemes give greater influence to closer observations (Gou, Xiong, and Kuang 2011). These approaches extend naturally to posterior probability estimation by specialized class frequencies or weighted contributions. The resulting probability estimates influence not only classification accuracy but also probabilistic loss metrics and calibration behaviour, which are central to reliable uncertainty quantification [Mallah and Orwell (2013)} (Holmes and Adams 2002).

3.2.5 Algorithmic Properties

Memory usage in kNN scales linearly with the size of the training set, as all observations must be stored (Bidye, Borkar, and Rakesh 2025). Prediction-time complexity is dominated by distance computations between the query instance and stored samples, making naïve implementations computationally expensive. Although specialized techniques exist, their effectiveness diminishes in high-dimensional spaces (KR et al. 2025). While the focus here is on classification, the same framework extends naturally to regression via local averaging of target values (Loeloe et al. 2025).

3.3 Package-Based Implementations

Package-based implementations of k-NN are widely adopted due to their robustness, efficiency, and integration within established machine-learning ecosystems (Pedregosa et al. 2011). Mature libraries provide standardised application programming interfaces (APIs) and consistent output formats, which support reproducibility and facilitate systematic comparison across experiments (Buitinck et al. 2013). To address the computational cost of neighbour search, these implementations often employ high-level optimisation strategies, including vectorised distance computations and specialised indexing structures (Ukey et al. 2023). While such abstractions improve performance and usability, they may reduce algorithmic transparency by concealing implementation details and design choices from the user. This inherent trade-off between performance and interpretability provides strong motivation for comparing package-based approaches with custom-built implementations.

4 Methodology

This section is specifically dedicated to describing the process of building both custom and package-based implementations of the k-Nearest Neighbours algorithm and carrying out the comparative study.

4.1 Experimental Design Overview

The primary purpose of comparing the performance of custom-built kNN with that of the package-based implementation is to make a contribution in the scientific field of supervised machine learning that is lacking the research comparing the performance of custom-built or adapted implementation of non-probabilistic classifiers with that of already-implemented ones.

The comparison is carried out by performing a Wilcoxon Signed Rank and Monte-Carlo Permutation tests across 6 datasets. Each of the tests were performed for the following metrics: macro-averaged versions of F1, Sensitivity, Recall, Area Under the Receiving Operator Curve (ROC-AUC), Categorical Cross-Entropy, Multi-Class Brier Score, and Expected Calibration Error (ECE).

4.2 kNN Implementations

4.2.1 Custom-Built kNN Implementation

The custom k-nearest neighbours (kNN) classifier was implemented from scratch to provide full control over algorithmic and implementation-level design choices. The implementation explicitly exposes the number of neighbours (k), the distance metric, and the voting strategy as tunable hyperparameters. Supported distance metrics include Euclidean and Manhattan distances, while classification decisions can be made using either uniform voting or distance-weighted voting. The model follows the standard instance-based learning paradigm, with no explicit training phase beyond storing the training data. This design prioritises transparency and reproducibility, enabling precise analysis of how implementation details influence performance, probabilistic behaviour, and computational characteristics.

4.2.2 Package-Based kNN Implementation

The package-based kNN classifier was implemented using a widely adopted machine learning library and served as a reference implementation in this study. It was configured to expose the same set of hyperparameters as the custom model, namely the number of neighbours (k), the distance metric, and the voting strategy. Default settings unrelated to these parameters were left unchanged to reflect typical usage. Care was taken to ensure that data preprocessing, train–test splits, and evaluation procedures were identical to those used for the custom implementation. This alignment enables a controlled comparison focused on implementation-level behaviour rather than differences arising from algorithmic configuration.

4.3 Datasets Used Throughout the Study

As mentioned, for comparing the two implementations of the same model, 6 distinct datasets were used. The summary of the structure of the datasets used throughout the comparative study are shown in the table below, highlighting the number of observations, categorical, and continuous features, etc. Please see Table 1.

Table 1: Dataset characteristics

Name	Obs.	Cat.	Num.
Iris	151	0	4
Wine	178	0	13
Digits	1797	0	64
Breast Cancer	569	0	29
Mushroom	8124	22	0
Zoo	101	2	14

4.4 Hyperparameter Grid and Experimental Design

4.4.1 Hyperparameter Space

The experimental comparison was conducted over a fixed, explicitly defined hyperparameter grid to allow a systematic and controlled assessment of both kNN implementations. Rather than performing

hyperparameter optimisation, the study evaluates a representative range of commonly used kNN settings. The number of neighbours k was varied across ten odd values from 3 to 22, capturing both highly local and more smoothed decision behaviour while reducing the likelihood of voting ties. Two distance metrics were considered: Euclidean distance and Manhattan distance, representing different geometric assumptions. Additionally, both uniform and distance-weighted voting schemes were evaluated. This design yields a diverse yet interpretable set of shared configurations.

4.4.2 Train-Test Splits and Experimental Consistency

The experimental comparison between the two implementations of models was performed using two separate split ratios for training and testing sets. More specifically, 70-30 and 80-20 split ratios were used, allowing us to observe the change of the generalisation behaviours of both models when it is exposed to more versus less training data.

In case of 70-30 split, 70% of data is dedicated to training, while the remaining 30% is allocated to testing the generalisation of the model. Similarly, in case of a 80-20 split, 80% of data is for training, while the rest 20% is for testing.

4.5 Evaluation Metrics

This subsection describes the set of evaluation criteria used to compare the custom-built and package-based kNN implementations. The selected metrics are intended to capture complementary aspects of model behaviour, including classification performance, probabilistic quality, calibration, and computational efficiency. All metrics are computed in a paired manner for both implementations under identical experimental conditions.

4.5.1 Performance and Discrimination Metrics

To evaluate classification performance, several macro-averaged metrics are employed. Macro-averaged F1-score, recall, and sensitivity are used to assess predictive quality across all classes by giving equal weight to each class, irrespective of class frequency. This choice is particularly important in multi-class settings and in datasets exhibiting class imbalance, where micro-averaged metrics may be dominated by majority classes. In addition, macro-averaged ROC–AUC is included as a threshold-independent measure of discriminative ability, summarising how well the predicted class probabilities separate each class from the others in a one-vs-rest formulation.

4.5.2 Probabilistic and Calibration Metrics

Beyond hard classification performance, the quality of predicted probabilities is assessed using probabilistic loss and calibration metrics. Categorical cross-entropy is used to quantify the divergence between the predicted probability distribution and the true class labels, penalising confident but incorrect predictions. The multi-class Brier score provides a complementary measure by capturing the mean squared error between predicted probabilities and one-hot encoded labels, offering sensitivity to both calibration and refinement. Expected Calibration Error (ECE) is used to directly assess how well predicted probabilities align with observed empirical frequencies, using a bin-based approximation to summarise miscalibration across the probability range.

4.5.3 Computational and Memory Metrics

To evaluate computational efficiency, multiple time-related metrics are recorded throughout the experimental pipeline. These include the time required to load each dataset, the total time consumed by 5-fold cross-validation, the training time during the fitting procedure, and the time required for testing and prediction. In addition to execution time, memory efficiency is assessed by measuring the peak RAM usage during model training. Together, these metrics provide insight into the practical trade-offs between implementation transparency and computational performance.

4.6 Cross-Validation and Testing Protocol

4.6.1 Cross-Validation Procedure

Cross-validation is used exclusively to estimate expected model performance under repeated sampling, rather than for hyperparameter optimisation. It provides variance-reduced estimates of predictive, probabilistic, and calibration metrics while mitigating sensitivity to a single data split. A 5-fold cross-validation scheme is applied uniformly to both kNN implementations, ensuring fair comparison. For each dataset, the training set is divided into five folds, with four used for training and one for validation in each iteration. Metrics are computed per fold and averaged to obtain a single cross-validated estimate per configuration.

4.6.2 Held-Out Test Evaluation

In addition to cross-validation, all models are evaluated on a held-out test set that remains entirely unseen during training and cross-validation. This evaluation provides an estimate of out-of-sample generalisation performance under a realistic deployment scenario. Test metrics are computed using the final model fitted on the full training set corresponding to each split. Reporting both cross-validated and test results allows differences between internal performance estimation and true generalisation behaviour to be examined.

4.6.3 Consistency and Separation of Evaluation Stages

To avoid information leakage, data preprocessing steps such as feature scaling are fitted exclusively on the training data and then applied to validation folds and test sets. Cross-validation and test evaluation are treated as distinct stages, with no reuse of test information during model fitting or metric estimation. This strict separation ensures that reported results reflect genuine generalisation differences between the two kNN implementations rather than artefacts of the evaluation protocol.

4.7 Dataset-Wise Experimental Repetition

4.7.1 Motivation for Dataset-Specific Evaluation

All experiments were conducted independently for each dataset rather than pooling results across datasets. This design choice reflects the fact that kNN performance and computational behaviour are strongly influenced by dataset characteristics such as dimensionality, class structure, sample size, and feature scaling. Treating datasets separately avoids masking heterogeneity through aggregation and allows conclusions to be drawn at the dataset level, which is particularly important when comparing implementations rather than abstract algorithms.

4.7.2 Repetition of the Full Experimental Pipeline

For each of the six datasets, the entire experimental pipeline was executed from start to finish. This included data loading and preprocessing, application of both train–test split strategies, execution of cross-validation and held-out testing, collection of performance, loss, time, and memory metrics, and storage of results for subsequent analysis. All hyperparameter combinations were evaluated independently within each dataset, ensuring that comparisons between implementations were always made under identical conditions.

4.7.3 Dataset-Specific Result Structuring

Results were organised on a per-dataset basis, with separate tables constructed for cross-validated metrics, test metrics, computational time measurements, and memory usage. This structure facilitates direct comparison between implementations within the same dataset and supports dataset-specific statistical testing. By preserving dataset-level granularity, the analysis remains sensitive to consistent patterns as well as dataset-dependent differences.

4.7.4 Implications for Statistical Inference

Conducting experiments independently for each dataset enables the application of paired statistical tests within each dataset context. This approach strengthens the validity of inference by avoiding assumptions of exchangeability across datasets and ensures that statistical significance reflects genuine performance differences rather than artefacts of aggregation.

5 Results

This section of the report is designed to present the results of the statistical tests performed for evaluating the difference between custom-built and package-based implementations of the kNN algorithm in the context of multi-class classification.

In terms of structure, this section is divided into several subsections, each discussing and presenting the results of statistical tests performed on metrics related to memory, performance, loss, and time.

Keeping the academic tone of this report concise and not passing the maximum page limit of 13 pages, it was decided that this section will not include interpretations of the results gathered from performing statistical tests. Instead, for more comprehensive metrics, including performance, loss, and time, the section will only present the results of the statistical tests performed for the UCI Iris dataset.

The rest of the results will be presented in the Appendix section of the report, with the interpretations provided in corresponding subsections of the Discussion section.

5.1 Memory-Related Statistical Results Across Datasets

This subsection is dedicated to presenting the results of statistical tests on memory-related metric across all datasets.

The only metric measured for memory for both the custom and package-based implementations of the kNN algorithm is the peak of Random Access Memory (RAM) usage in Megabytes during the process of training the models, i.e. applying the `.fit` command, giving the training features and targets as parameters.

The memory-related metric is tested for 70-30 and 80-20 train-test splits separately for all datasets in order to accentuate the influence of the size of the datasets as well as that of the split in the training and testing datasets.

The results of both statistical tests for all 6 datasets and 2 splits are shown in the table below. See table 2.

Table 2: Memory-related statistical test results

Dataset	Split	Perm.	p	Wilc.	p
iris	70/30	$-9.77 \cdot 10^{-5}$	0.4985	0	0.1587
	80/20	-0.001563	0.5006	0	0.1587
wine	70/30	0.002637	1	36	0.9947
	80/20	0.001563	0.8126	7	0.7693
digits	70/30	-0.03057	**0.00502**	142	**0.002292**
	80/20	-0.04619	**0**	66	$\mathbf{1.86 \cdot 10^{-6}}$
breast cancer	70/30	-0.0008789	0.06266	1	**0.038441**
	80/20	-0.001563	**0.00208**	0	**0.003453**
mushroom	70/30	-0.1914	**0**	0	$\mathbf{1.77 \cdot 10^{-8}}$
	80/20	-0.1472	**0**	0	$\mathbf{2.60 \cdot 10^{-8}}$
zoo	70/30	$-9.77 \cdot 10^{-5}$	0.5006	0	0.1587
	80/20	$-9.77 \cdot 10^{-5}$	0.5006	0	0.1587

5.2 Cross-Validated and Test Performance/Loss Metrics

This subsection is for providing an overview of the results gathered as a result of performing statistical tests on performance and loss metrics gathered during the processes of cross-validating and testing the models on all datasets.

As mentioned in the methodology section, the statistical testing and experimental design procedure included two statistical tests: Wilcoxon Signed Rank Test as well as a Monte-Carlo Permutation Test, both at a confidence level of 95%.

The results of the statistical tests for the mentioned dataset are presented in the table below. See Table 3.

Table 3: Cross-validated and test performance and loss metrics (Iris)

Metric	Prot.	Split	Perm.	p	Wilc.	p
Macro-F1	CV	70/30	0.00034	0.434	439	0.351
	CV	80/20	0.0301	**0**	820	$\mathbf{1.84 \cdot 10^{-8}}$
	Test	70/30	$-1.37 \cdot 10^{-10}$	0.997	292	0.946
	Test	80/20	0	1	0	1
Macro-Recall	CV	70/30	-0.00021	0.547	459	0.257
	CV	80/20	0.0118	**0**	793	$\mathbf{1.36 \cdot 10^{-7}}$
	Test	70/30	$1.55 \cdot 10^{-10}$	$\mathbf{4 \cdot 10^{-5}}$	663	$\mathbf{2.71 \cdot 10^{-4}}$
	Test	80/20	0	1	0	1
Macro-Sensitivity	CV	70/30	-0.00021	0.547	459	0.257
	CV	80/20	0.0118	**0**	793	$\mathbf{1.36 \cdot 10^{-7}}$
	Test	70/30	$1.55 \cdot 10^{-10}$	$\mathbf{4 \cdot 10^{-5}}$	663	$\mathbf{2.71 \cdot 10^{-4}}$
	Test	80/20	0	1	0	1
ROC-AUC	CV	70/30	-0.00186	1	65	1
	CV	80/20	-0.00120	1	151	1
	Test	70/30	$-6.48 \cdot 10^{-11}$	0.935	300	0.932
	Test	80/20	0	1	0	1
Cross-Entropy	CV	70/30	0.00104	0.208	413	0.519
	CV	80/20	0.0317	**0**	1	$\mathbf{2 \cdot 10^{-8}}$
	Test	70/30	$1.47 \cdot 10^{-8}$	**0**	132	$\mathbf{9.58 \cdot 10^{-5}}$
	Test	80/20	$2 \cdot 10^{-8}$	**0**	0	**0**
Brier	CV	70/30	$5.87 \cdot 10^{-5}$	0.475	460	0.751
	CV	80/20	0.0168	**0**	1	$\mathbf{2 \cdot 10^{-8}}$
	Test	70/30	$1.01 \cdot 10^{-8}$	**0**	110	$\mathbf{2.84 \cdot 10^{-5}}$
	Test	80/20	$1.5 \cdot 10^{-10}$	**0**	0	**0**
ECE	CV	70/30	-0.00342	0.955	531	0.949
	CV	80/20	0.00407	**0.0159**	286	**0.0485**
	Test	70/30	$7.53 \cdot 10^{-9}$	$\mathbf{2 \cdot 10^{-5}}$	178	$\mathbf{9.3 \cdot 10^{-4}}$
	Test	80/20	$1.27 \cdot 10^{-8}$	**0**	0	**0**

5.3 Computational Time Analysis

This subsection is dedicated to present and report the results of the statistical tests that were done on the time-related metrics for both the custom and package-based metrics across two train-test split ratios.

As in the previous subsection, this section only presents the results of the statistical tests done on the representative, i.e. the UCI Iris dataset.

The following computational time-related metrics were gathered and tested: the time taken to load the dataset, to perform a 5-fold cross-validation, training, and testing the models. As before, the two statistical tests were performed on the confidence level of 95 percent, i.e. the results are considered significant if the p-value is lower than 0.05.

The results of the statistical tests on time-related metrics for the UCI Iris dataset are presented in the table below. See Table Z.

Table 4: Time-related statistical test results (Iris)

Metric	Split	Perm.	p	Wilc.	p
Load data	70/30	-0.03126	**0**	0	$\mathbf{1.33 \cdot 10^{-10}}$
	80/20	0.09113	1	820	1
Cross-val.	70/30	0.004504	0.9998	587	0.9915
	80/20	0.002148	0.9338	412	0.5134
Train	70/30	0.000371	0.7385	154	**0.0002971**
	80/20	0.000257	0.4983	40	$\mathbf{3.41 \cdot 10^{-7}}$
Test	70/30	0.000619	0.7379	295	0.0619
	80/20	0.001167	0.8759	292	0.005713

6 Discussion

As mentioned in the Results section of the report, the main aim of this section is to present valuable insights and interpret the results gathered from both Monte-Carlo Permutation and Wilcoxon Signed Rank Tests across all performance, loss, memory-, and time complexity-related metrics.

While the results of tests for individual datasets are provided in the Appendix section, their interpretations are provided here.

6.1 Predictive Performance and Discrimination

6.1.1 Cross-Validation and Testing Results

Cross-validation results indicate that the two kNN implementations exhibit broadly similar discriminative performance across datasets, with most macro-averaged metrics showing only minor numerical differences. Detailed, dataset-specific results are reported in the appendix. Rather than consistent performance gaps, observed differences tend to vary with the chosen train–test split and dataset size. In particular, larger training proportions occasionally amplify small implementation-level effects, while smaller training sets yield more variable outcomes under resampling.

Test-set evaluations largely reflect these patterns. Classification performance remains comparable in most cases, but differences in loss-based and calibration-related metrics become more apparent for certain datasets and split ratios. This suggests that variations between implementations are more strongly influenced by dataset characteristics and training set size than by systematic differences in predictive accuracy, highlighting the complementary roles of cross-validation and held-out testing in revealing such effects.

6.2 Probabilistic Quality and Calibration

Probabilistic quality and calibration were assessed using categorical cross-entropy, the multi-class Brier score, and Expected Calibration Error, which directly evaluate the reliability of predicted class probabilities rather than label accuracy. Detailed, dataset-level results are provided in the appendix. Across datasets, these probabilistic metrics exhibit greater sensitivity to implementation differences than discrimination-based measures, indicating that probability estimation is more strongly affected by implementation-level design choices.

Under cross-validation, statistically significant differences in cross-entropy, Brier score, and calibration error arise for several datasets, particularly under the 80-20 split, where larger training sets tend to accentuate such effects. Test-set results generally reflect the same tendencies with reduced variability, suggesting that while some differences attenuate on unseen data, calibration behaviour remains a consistent source of divergence. Overall, these results highlight that similar classification accuracy can mask meaningful differences in probabilistic reliability.

6.3 Computational Time Behaviour

Computational time behaviour was evaluated using dataset loading, cross-validation, training, and testing times, with detailed dataset-level results reported in the appendix. Loading time occasionally shows statistically significant differences, reflecting implementation-specific data handling rather than algorithmic complexity. Cross-validation time generally exhibits similar scaling behaviour across implementations, particularly for smaller datasets. In contrast, training and testing times display greater sensitivity to dataset size and train–test split, with larger training sets sometimes amplifying runtime differences. These findings indicate that, despite sharing the same algorithmic structure, implementation-level design choices can lead to measurable variations in computational efficiency across different stages of execution.

6.4 Memory Usage Characteristics

Memory usage was evaluated by measuring peak RAM consumption during the model fitting stage for each implementation. As kNN is an instance-based learner, memory requirements primarily reflect how training data and auxiliary structures are stored rather than the cost of parameter optimisation. Across datasets, memory usage results show a mix of statistically significant and non-significant differences between implementations. In several cases, both permutation and Wilcoxon tests indicate comparable memory footprints, suggesting similar data storage strategies. In other datasets, however, consistent and significant differences emerge, highlighting the impact of implementation-specific design choices on memory efficiency. These findings indicate that memory behaviour is not solely determined by dataset size, but also by how neighbourhood computations and internal representations are managed.

6.5 Influence of Dataset Characteristics

The comparative results reveal that dataset characteristics play a substantial role in shaping observed differences between the two kNN implementations. Factors such as sample size, feature dimensionality, class balance, and class overlap influence not only predictive performance but also calibration behaviour, computational time, and memory usage. Smaller or well-separated datasets tend to produce highly similar outcomes across implementations, whereas larger or more complex datasets more frequently expose implementation-level differences, particularly in loss-based, calibration, and resource-related metrics. Additionally, changes in train–test split ratios interact with dataset size to affect both stability and statistical significance of results. These observations highlight that the practical impact of implementation choices cannot be assessed independently of data properties, and that conclusions drawn from a single dataset may not generalise across different problem settings.

6.6 Interpretation of Statistical Evidence

The statistical analyses combine non-parametric and permutation-based inference to characterise differences between the two kNN implementations. The Wilcoxon signed-rank test assesses median differences under minimal distributional assumptions, while the Monte Carlo permutation test evaluates the likelihood of observed effects under random relabelling. Concordance between the two tests strengthens confidence that detected differences reflect systematic implementation effects rather than sampling noise. In many comparisons, neither test rejects the null hypothesis, suggesting that numerical differences are not statistically meaningful. When both tests yield low p-values across multiple metrics or splits, the evidence supports consistent implementation-level differences. Divergent test outcomes are interpreted cautiously, as they may reflect marginal effects or differences in test sensitivity. Overall, the statistical evidence contextualises metric-level differences rather than implying categorical superiority.

7 Conclusion

This study systematically evaluated the behavior of a custom KNN implementation against a well-established library-based counterpart across multiple datasets, data splits, and evaluation dimensions. By combining cross-validated performance metrics, test-set outcomes, calibration measures, and detailed timing analyses, the results provide a comprehensive view of both predictive quality and computational efficiency.

Across datasets, the performance-oriented metrics—such as macro-averaged F1, recall, sensitivity, and ROC–AUC—largely exhibited no statistically significant differences on held-out test sets. This indicates that, despite implementation-level differences, both approaches converge to comparable predictive behavior when evaluated under identical data partitions. In contrast, several cross-validated performance and loss metrics revealed statistically significant differences, particularly for probabilistic losses and calibration-related measures. These findings suggest that subtle distinctions in prediction aggregation, numerical precision, or probability estimation may become more apparent under repeated resampling, even when final test performance remains similar.

Calibration and loss-based metrics consistently showed stronger sensitivity to implementation differences than threshold-based classification scores. This highlights the importance of evaluating models beyond accuracy-driven metrics, especially in applications where probabilistic reliability is critical. However, the observed differences did not uniformly favor one approach across all datasets, underscoring the dataset-dependent nature of such effects.

The timing analysis revealed the most pronounced and consistent differences between implementations. In particular, cross-validation and test-time execution exhibited statistically significant disparities in several cases, reflecting differences in internal optimizations, memory handling, and computational overhead. Training and data-loading times, by contrast, were generally comparable or showed limited practical relevance despite occasional statistical significance.

Overall, the findings demonstrate that while predictive performance between the two implementations is largely equivalent in practical terms, meaningful differences emerge in calibration behavior and computational efficiency. These results emphasize that implementation choices can have tangible effects beyond raw accuracy, particularly in large-scale or time-sensitive workflows. Consequently, the selection of a machine learning implementation should be informed not only by predictive performance, but also by considerations of calibration quality, reproducibility under resampling, and computational cost.

8 References

9 Appendix

9.1 Appendix 1: Results of statistical tests on Cross-validation and Testing Results

Table 5: Statistical test results for cross-validation and test performance and loss metrics (UCI Wine)

Metric	Prot.	Split	Perm.	Perm. p	Wilc.	Wilc. p
Macro-F1	CV	70/30	0.001842	0.1486	358	0.7603
	CV	80/20	-0.008660	1	128	1
	Test	70/30	0.0005275	0.2476	5	0.2113
	Test	80/20	0	1	0	1
Macro-Recall	CV	70/30	-0.002809	0.9934	188	0.9986
	CV	80/20	-0.01071	1	24	1
	Test	70/30	0.0003968	0.4971	4	0.3864
	Test	80/20	0	1	0	1
Macro-Sensitivity	CV	70/30	-0.002809	0.9934	188	0.9986
	CV	80/20	-0.01071	1	24	1
	Test	70/30	0.0003968	0.4971	4	0.3864
	Test	80/20	0	1	0	1
ROC-AUC	CV	70/30	0.0004404	0.1238	301	0.7792
	CV	80/20	-0.0007611	0.9999	85	1
	Test	70/30	$-6.48 \cdot 10^{-11}$	0.9348	300	0.9317
	Test	80/20	0	1	0	1
Cross-Entropy	CV	70/30	0.052455	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	CV	80/20	-0.0007188	0.7563	325	0
	Test	70/30	$8.41 \cdot 10^{-10}$	**0**	18	$\mathbf{3.34 \cdot 10^{-5}}$
	Test	80/20	$9.4 \cdot 10^{-10}$	**0**	0	$\mathbf{4.79 \cdot 10^{-5}}$
Brier	CV	70/30	0.008139	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	CV	80/20	-0.001305	0.9430	477	1
	Test	70/30	$5.37 \cdot 10^{-10}$	**0**	16	$\mathbf{2.69 \cdot 10^{-5}}$
	Test	80/20	$6.63 \cdot 10^{-10}$	**0**	0	$\mathbf{4.79 \cdot 10^{-5}}$
ECE	CV	70/30	0.009011	$\mathbf{2 \cdot 10^{-5}}$	69	$\mathbf{2.36 \cdot 10^{-6}}$
	CV	80/20	0.005812	$\mathbf{1 \cdot 10^{-4}}$	286	$\mathbf{6.74 \cdot 10^{-4}}$
	Test	70/30	$-9.08 \cdot 10^{-5}$	0.5029	59	**0.001609**
	Test	80/20	$7.11 \cdot 10^{-10}$	**0**	0	$\mathbf{4.79 \cdot 10^{-5}}$

Table 6: Statistical test results for cross-validation and test performance and loss metrics (UCI Digits)

Metric	Prot.	Split	Perm.	p	Wilc.	p
Macro-F1	CV	70/30	-0.00630	1	0	1
	CV	80/20	-0.00014	0.6163	441	0.3409
	Test	70/30	-0.00118	1	1	0.9998
	Test	80/20	-0.00121	1	2	0.9993
Macro-Recall	CV	70/30	-0.00779	1	0	1
	CV	80/20	-0.00074	0.9336	330	0.8604
	Test	70/30	-0.00117	1	0	0.9997
	Test	80/20	-0.00119	1	0	0.9990
Macro-Sensitivity	CV	70/30	-0.00779	1	0	1
	CV	80/20	-0.00074	0.9336	330	0.8604
	Test	70/30	-0.00117	1	0	0.9997
	Test	80/20	-0.00119	1	0	0.9990
ROC–AUC	CV	70/30	0.00017	**0.02224**	602	**0.005026**
	CV	80/20	$2.67 \cdot 10^{-5}$	0.3310	423	0.4333
	Test	70/30	$-7.65 \cdot 10^{-7}$	0.7588	89	0.7310
	Test	80/20	$-1.30 \cdot 10^{-6}$	0.7615	60	0.7895
Cross-Entropy	CV	70/30	0.02120	$\mathbf{2.0 \cdot 10^{-5}}$	133	**0.0001010**
	CV	80/20	0.00385	0.1133	289	0.05265
	Test	70/30	$4.65 \cdot 10^{-10}$	**0**	194.5	**0.001924**
	Test	80/20	$4.08 \cdot 10^{-10}$	**0**	150	**0.0004152**
Brier	CV	70/30	-0.00386	1	820	1
	CV	80/20	0.00123	**0**	27	$\mathbf{1.36 \cdot 10^{-7}}$
	Test	70/30	$2.96 \cdot 10^{-10}$	**0**	179.5	**0.001691**
	Test	80/20	$2.52 \cdot 10^{-10}$	**0**	151.5	**0.0004479**
ECE	CV	70/30	0.00252	**0**	41	$\mathbf{3.65 \cdot 10^{-7}}$
	CV	80/20	0.00091	**0.02564**	258	**0.02086**
	Test	70/30	0.00065	**0.00172**	99	**0**
	Test	80/20	0.00021	0.2058	129	**0.0001388**

Table 7: Statistical test results for cross-validation and test performance and loss metrics (UCI Breast Cancer)

Metric	Prot.	Split	Perm.	p	Wilc.	p
Macro-F1	CV	70/30	0.001979	0.01894	572	**0.01497**
	CV	80/20	-0.001224	0.9644	270	0.9705
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Macro-Recall	CV	70/30	0.001971	**0.02916**	537	**0.04451**
	CV	80/20	-0.002608	0.9998	155	0.9997
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Macro-Sensitivity	CV	70/30	0.001971	**0.02916**	537	**0.04451**
	CV	80/20	-0.002608	0.9998	155	0.9997
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
ROC–AUC	CV	70/30	0.004672	**0**	820	$\mathbf{1.85 \cdot 10^{-8}}$
	CV	80/20	0.004057	**0**	816	$\mathbf{2.51 \cdot 10^{-8}}$
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Cross-Entropy	CV	70/30	0.06942	**0**	50	$\mathbf{6.75 \cdot 10^{-7}}$
	CV	80/20	0.04397	**0**	59	$\mathbf{1.23 \cdot 10^{-6}}$
	Test	70/30	$3.67 \cdot 10^{-10}$	**0**	0	$\mathbf{4.79 \cdot 10^{-5}}$
	Test	80/20	$1.16 \cdot 10^{-10}$	**0.00278**	25	**0.001499**
Brier	CV	70/30	0.003630	**0**	77	$\mathbf{3.93 \cdot 10^{-6}}$
	CV	80/20	0.001897	$\mathbf{6.00 \cdot 10^{-5}}$	126	$\mathbf{6.93 \cdot 10^{-5}}$
	Test	70/30	$2.74 \cdot 10^{-10}$	**0**	0	$\mathbf{4.79 \cdot 10^{-5}}$
	Test	80/20	$5.71 \cdot 10^{-11}$	**0.03672**	41	**0.008879**
ECE	CV	70/30	0.005158	**0**	100	$\mathbf{1.59 \cdot 10^{-5}}$
	CV	80/20	0.001104	0.1047	290	0.08249
	Test	70/30	$1.69 \cdot 10^{-10}$	$\mathbf{3.00 \cdot 10^{-4}}$	15	**0.0004170**
	Test	80/20	$1.36 \cdot 10^{-10}$	**0.00094**	29	**0.002412**

Table 8: Statistical test results for cross-validation and test performance and loss metrics (UCI Mushroom)

Metric	Prot.	Split	Perm.	p	Wilc.	p
Macro-F1	CV	70/30	$5.31 \cdot 10^{-5}$	0.0881	261	0.2823
	CV	80/20	$-1.58 \cdot 10^{-4}$	0.9972	164	0.9977
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Macro-Recall	CV	70/30	$4.07 \cdot 10^{-5}$	0.1590	242	0.4265
	CV	80/20	$-1.64 \cdot 10^{-4}$	0.9985	155	0.9985
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Macro-Sensitivity	CV	70/30	$4.07 \cdot 10^{-5}$	0.1590	242	0.4265
	CV	80/20	$-1.64 \cdot 10^{-4}$	0.9985	155	0.9985
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
ROC–AUC	CV	70/30	$-9.23 \cdot 10^{-7}$	0.9998	32	0.9994
	CV	80/20	$3.05 \cdot 10^{-6}$	**0**	190	$\mathbf{7.13 \cdot 10^{-5}}$
	Test	70/30	0	1	0	1
	Test	80/20	0	1	0	1
Cross-Entropy	CV	70/30	$1.44 \cdot 10^{-4}$	**0**	16	$\mathbf{6.14 \cdot 10^{-8}}$
	CV	80/20	$-7.44 \cdot 10^{-5}$	0.8556	507	0.9050
	Test	70/30	$3.71 \cdot 10^{-10}$	**0**	0	$\mathbf{1.07 \cdot 10^{-4}}$
	Test	80/20	$3.57 \cdot 10^{-10}$	**0**	0	$\mathbf{1.61 \cdot 10^{-4}}$
Brier	CV	70/30	$1.17 \cdot 10^{-4}$	**0**	8	$\mathbf{3.39 \cdot 10^{-8}}$
	CV	80/20	$-1.27 \cdot 10^{-4}$	0.9943	618	0.9975
	Test	70/30	$2.47 \cdot 10^{-10}$	**0**	0	$\mathbf{1.07 \cdot 10^{-4}}$
	Test	80/20	$2.27 \cdot 10^{-10}$	**0**	0	$\mathbf{1.61 \cdot 10^{-4}}$
ECE	CV	70/30	$7.13 \cdot 10^{-5}$	0.0922	161	$\mathbf{4.19 \cdot 10^{-4}}$
	CV	80/20	$1.23 \cdot 10^{-4}$	**0.0107**	279	**0.0397**
	Test	70/30	$2.55 \cdot 10^{-10}$	**0**	0	$\mathbf{1.07 \cdot 10^{-4}}$
	Test	80/20	$2.59 \cdot 10^{-10}$	**0**	0	$\mathbf{1.61 \cdot 10^{-4}}$

Table 9: Statistical test results for cross-validation and test performance and loss metrics (UCI Zoo)

Metric	Prot.	Split	Perm.	p	Wilc.	p
Macro-F1	CV	70/30	-0.01614	0.9370	337	0.8384
	CV	80/20	-0.07868	1	4	1
	Test	70/30	-0.01534	0.9861	6	0.9782
	Test	80/20	-0.01915	0.9538	3	0.9537
Macro-Recall	CV	70/30	-0.01515	0.9472	346	0.8071
	CV	80/20	-0.07273	1	5	1
	Test	70/30	-0.01310	1	0	0.9903
	Test	80/20	-0.01786	1	0	0.9786
Macro-Sensitivity	CV	70/30	-0.01515	0.9472	346	0.8071
	CV	80/20	-0.07273	1	5	1
	Test	70/30	-0.01310	1	0	0.9903
	Test	80/20	-0.01786	1	0	0.9786
ROC-AUC	CV	70/30	–	–	–	–
	CV	80/20	–	–	–	–
	Test	70/30	-0.006788	0.9945	11	0.9876
	Test	80/20	0.0001786	0.3755	4	0.3946
Cross-Entropy	CV	70/30	0.1795	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	CV	80/20	0.01052	0.3208	294	0.06027
	Test	70/30	$4.46 \cdot 10^{-8}$	**0**	153	**0.001408**
	Test	80/20	$2.88 \cdot 10^{-8}$	**0**	105	**0.0005140**
Brier	CV	70/30	0.02778	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	CV	80/20	0.007933	0.01716	296	0.06356
	Test	70/30	$5.99 \cdot 10^{-9}$	**0**	153	**0.001408**
	Test	80/20	$7.47 \cdot 10^{-9}$	**0**	105	**0.0005143**
ECE	CV	70/30	0.009673	**0.00478**	232	**0.008520**
	CV	80/20	0.01547	**0.00074**	226	**0.006822**
	Test	70/30	0.001919	0.1578	152	**0.001340**
	Test	80/20	0.004035	**0.03164**	75	$\mathbf{7.37 \cdot 10^{-5}}$

9.2 Appendix 2: Results of Statistical Testing on Time-Related Metrics

Table 10: Time-related statistical test results (UCI Wine)

Metric	Split	Perm.	p	Wilc.	p
Load Data	70/30	0.07133	1	820	1
	80/20	0.07634	1	820	1
Cross-Validate	70/30	0.00410	0.9996	574	0.9865
	80/20	0.00616	0.9994	619	0.9976
Train	70/30	0.00033	0.4986	117	$\mathbf{4.22 \cdot 10^{-5}}$
	80/20	0.00109	0.9784	259	**0.02154**
Test	70/30	0.00050	0.6871	354	0.2278
	80/20	0.00145	0.9420	402	0.4599

Table 11: Time-related statistical test results (UCI Digits)

Metric	Split	Perm.	p	Wilc.	p
Load	70/30	0.06061	1	820	1
	80/20	0.06358	1	820	1
Cross-Validate	70/30	-0.1989	**0**	1	$\mathbf{2.00 \cdot 10^{-8}}$
	80/20	-0.2928	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
Train	70/30	0.0001483	0.8032	355	0.2319
	80/20	0.0001761	0.7488	353	0.2238
Test	70/30	-0.12288	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	80/20	-0.09359	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$

Table 12: Time-related statistical test results (UCI Breast Cancer)

Metric	Split	Perm.	p	Wilc.	p
Load Data	70/30	0.08046	1	820	1
	80/20	0.06145	1	820	1
Cross-Validate	70/30	0.01509	0.9998	640	0.9990
	80/20	0.01684	0.9999	666	0.9997
Train	70/30	$5.61 \cdot 10^{-5}$	0.5235	154	**0.000297**
	80/20	0.0003181	0.9862	414	0.5241
Test	70/30	-0.004626	**0**	103	$\mathbf{1.90 \cdot 10^{-5}}$
	80/20	0.001789	0.9890	568	0.9834

Table 13: Time-related statistical test results (UCI Mushroom)

Metric	Split	Perm.	p	Wilc.	p
Load Data	70/30	-0.01917	**0**	0	$\mathbf{1.33 \cdot 10^{-10}}$
	80/20	0.007664	1	820	1
Cross-Validate	70/30	-2.8155	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	80/20	-3.3571	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
Train	70/30	-0.0001415	0.2516	316	0.10442
	80/20	-0.0001882	0.2226	181	**0.00107**
Test	70/30	-1.5627	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$
	80/20	-1.0328	**0**	0	$\mathbf{1.85 \cdot 10^{-8}}$

Table 14: Time-related statistical test results (UCI Zoo)

Metric	Split	Perm.	p	Wilc.	p
Load Data	70/30	-0.006118	**0**	0	$\mathbf{1.33 \cdot 10^{-10}}$
	80/20	0.0006944	1	820	1
Cross-Validate	70/30	0.04252	1	820	1
	80/20	0.02039	1	739	0.9999
Train	70/30	0.0001418	0.9110	259	**0.02154**
	80/20	0.0002077	0.7504	114	$\mathbf{3.56 \cdot 10^{-5}}$
Test	70/30	0.009525	1	820	1
	80/20	0.008664	1	819	1

Abu Alfeilat, Haneen Arafat, Ahmad BA Hassanat, Omar Lasassmeh, Ahmad S Tarawneh, Mahmoud Bashir Alhasanat, Hamzeh S Eyal Salman, and VB Surya Prasath. 2019. "Effects of Distance Measure Choice on k-Nearest Neighbor Classifier Performance: A Review." *Big Data* 7 (4): 221–48.

Aggarwal, Charu C, Alexander Hinneburg, and Daniel A Keim. 2001. "On the Surprising Behavior of Distance Metrics in High Dimensional Space." In *International Conference on Database Theory*, 420–34. Springer.

Alshammari, Ali Hussein, Gergely Bencsik, and Almashhadani Hasnain Ali. 2026. "A Survey of Six Classical Classifiers, Including Algorithms, Methodological Characteristics, Foundational Variants, and Recent Advances." *Algorithms* 19 (1). https://doi.org/10.3390/a19010037.

Bidye, Pooja, Pradnya Borkar, and Nitin Rakesh. 2025. "High Performance GPU Implementation of KNN Algorithm: A Review." *MethodsX*, 103633.

Buitinck, Lars, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, et al. 2013. "API Design for Machine Learning Software: Experiences from the Scikit-Learn Project." https://arxiv.org/abs/1309.0238.

Dritsas, Elias, and Maria Trigka. 2025. "Exploring the Intersection of Machine Learning and Big Data: A Survey." *Machine Learning and Knowledge Extraction* 7 (1). https://doi.org/10.3390/make7010013.

Ersozlu, Zara, Sona Taheri, and Inge Koch. 2024. "A Review of Machine Learning Methods Used for Educational Data." *Education and Information Technologies* 29 (16): 22125–45.

Gou, Jianping, Taisong Xiong, and Yin Kuang. 2011. "A Novel Weighted Voting for k-Nearest Neighbor Rule." *JCP* 6 (May): 833–40. https://doi.org/10.4304/jcp.6.5.833-840.

Guidotti, Dario, Laura Pandolfo, and Luca Pulina. 2025. "A Systematic Literature Review of Supervised Machine Learning Techniques for Predictive Maintenance in Industry 4.0." *IEEE Access* 13: 102479–504. https://doi.org/10.1109/ACCESS.2025.3578686.

Gweon, Hyukjun, Matthias Schonlau, and Stefan Steiner. 2019a. "The k Conditional Nearest Neighbor Algorithm for Classification and Class Probability Estimation." *PeerJ Computer Science* 5 (May): e194. https://doi.org/10.7717/peerj-cs.194.

Gweon, Hyukjun, Matthias Schonlau, and Stefan H Steiner. 2019b. "The k Conditional Nearest Neighbor Algorithm for Classification and Class Probability Estimation." *PeerJ Computer Science* 5: e194.

Halder, Rajib Kumar, Mohammed Nasir Uddin, Md Ashraf Uddin, Sunil Aryal, and Ansam Khraisat. 2024. "Enhancing k-Nearest Neighbor Algorithm: A Comprehensive Review and Performance Analysis of Modifications." *Journal of Big Data* 11 (1): 113.

Holmes, CC, and NM Adams. 2002. "A Probabilistic Nearest Neighbour Method for Statistical Pattern Recognition." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 64 (2): 295–306.

Karakullukcu, Erdinc. 2025. "Leveraging Convolutional Neural Networks for Image-Based Classification of Feature Matrix Data." *Expert Systems with Applications* 281: 127625.

KR, Madhavan, Hasan Kurban, Oguzhan M Kulekci, and Mehmet M Dalkilic. 2025. "Telescope Indexing for k-Nearest Neighbor Search Algorithms over High Dimensional Data & Large Data Sets." *Scientific Reports* 15 (1): 24788.

Loeloe, Mohammad Sadegh, Seyyed Mohammad Tabatabaei, Reyhane Sefidkar, Amir Houshang Mehrparvar, and Sara Jambarsang. 2025. "Boosting k-Nearest Neighbor Regression Performance for Longitudinal Data Through a Novel Learning Approach." *BMC Bioinformatics* 26 (1): 232.

Lu, Jiaxuan, and Hyukjun Gweon. 2025. "Random k Conditional Nearest Neighbor for High-Dimensional Data." *PeerJ Computer Science* 11: e2497.

Mallah, Charles, and James Orwell. 2013. "Probabilistic Classification from a k-Nearest-Neighbour Classifier." *Computational Research* 1 (October): 1–9. https://doi.org/10.13189/cr.2013.010101.

Mehdiyev, Nijat, Maxim Majlatow, and Peter Fettke. 2025. "Interpretable and Explainable Machine

Learning Methods for Predictive Process Monitoring: A Systematic Literature Review." *Artificial Intelligence Review* 58 (12): 378.

Mehta, Sumet, Xiangjun Shen, Jiangping Gou, and Dejiao Niu. 2018. "A New Nearest Centroid Neighbor Classifier Based on k Local Means Using Harmonic Mean Distance." *Information* 9 (9). https://doi.org/10.3390/info9090234.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *The Journal of Machine Learning Research* 12: 2825–30.

Ukey, Nimish, Zhengyi Yang, Binghao Li, Guangjian Zhang, Yiheng Hu, and Wenjie Zhang. 2023. "Survey on Exact kNN Queries over High-Dimensional Data Space." *Sensors* 23 (2). https://doi.org/10.3390/s23020629.